



Programmation parallèle pour le Machine Learning

Parallélisation des forêts aléatoires : benchmark adaptatif

Étudiants :

ALUCH Yasmine OUALY Ossama

Enseignant :

M. DUPRÉ Xavier

Parallélisation des forêts aléatoires : benchmark adaptatif

Résumé

Les forêts aléatoires se prêtent naturellement à la parallélisation, puisque les arbres sont indépendants pendant l'entraînement. Ce rapport étudie ce parallélisme avec scikit-learn et joblib, et propose un benchmark adaptatif qui cherche le nombre optimal de workers en testant moins de configurations qu'une grid search. Les mesures donnent un speedup réel de $4,21\times$ avec 7 workers, contre un idéal de $7\times$, ce qui correspond à une fraction parallèle effective d'environ 87–89% selon la manière de l'estimer. Le benchmark adaptatif retrouve un optimum proche en testant 37,5% de configurations en moins.

1 Description mathématique

1.1 Arbre de décision (CART)

L'algorithme CART (*Classification And Regression Trees*) partitionne récursivement l'espace des features en cherchant, à chaque nœud, le split qui réduit le plus une mesure d'impureté. La plus utilisée en classification est l'*indice de Gini*, qui mesure la probabilité de mal classer un échantillon tiré au hasard dans le nœud : si p_k est la proportion d'échantillons de la classe k dans le nœud S et K le nombre de classes,

$$G(S) = 1 - \sum_{k=1}^K p_k^2. \quad (1)$$

Cette quantité vaut 0 pour un nœud pur et $1 - 1/K$ quand les classes sont équidistribuées. L'*entropie de Shannon* est une alternative issue de la théorie de l'information :

$$H(S) = - \sum_{k=1}^K p_k \log_2(p_k). \quad (2)$$

Pour un split candidat qui divise S en S_L et S_R , le gain d'information vaut

$$\Delta G(S, j, t) = G(S) - \frac{|S_L|}{|S|} G(S_L) - \frac{|S_R|}{|S|} G(S_R), \quad (3)$$

où j est la feature choisie et t le seuil. CART teste tous les couples (j, t) et retient celui qui maximise ce gain : $(j^*, t^*) = \arg \max_{j, t} \Delta G(S, j, t)$. La récursion s'arrête à une profondeur maximale fixée à l'avance, ou quand $|S|$ passe sous un seuil `min_samples`. Un exemple d'arbre extrait de notre forêt est donné en annexe A.

1.2 Forêt aléatoire (Bagging + Random Subspace)

Un arbre seul a tendance à surapprendre. La forêt aléatoire de Breiman corrige ce défaut en combinant deux idées. La première est le *bagging* : pour $b = 1, \dots, B$, on tire un échantillon bootstrap D_b par tirage avec remise, avec $|D_b| = |D|$. Chaque arbre est entraîné sur son propre échantillon, ce qui décorrèle les modèles et réduit la variance globale. La seconde est la *Random Subspace Method* : à chaque nœud, on ne regarde qu'un sous-ensemble aléatoire de m features parmi p . En classification, le choix usuel est $m = \lfloor \sqrt{p} \rfloor$. Cette restriction empêche tous les arbres de choisir les mêmes features dominantes, ce qui les décorrèle davantage. La prédiction finale se fait par vote majoritaire :

$$\hat{y} = \arg \max_k \sum_{b=1}^B \mathbf{1}[\hat{y}_b(x) = k]. \quad (4)$$

1.3 Parallélisation et loi d'Amdahl

Les B arbres sont indépendants pendant l'entraînement : aucune communication n'est requise entre eux. C'est un problème *embarrassingly parallel*, qu'on peut distribuer sur n workers sans synchronisation pendant le calcul. Le speedup atteignable est borné par la *loi d'Amdahl* : si p est la fraction parallélisable et $1 - p$ la fraction séquentielle,

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}, \quad \lim_{n \rightarrow \infty} S(n) = \frac{1}{1-p}. \quad (5)$$

Même avec une infinité de cœurs, le speedup plafonne à $1/(1-p)$. L'*efficacité parallèle* $E(n) = S(n)/n$ mesure quel pourcentage du temps CPU sert effectivement : $E = 1$ signifie un speedup linéaire, et une efficacité qui chute traduit du temps perdu en overhead de coordination.

1.4 Benchmark adaptatif

Tester toutes les valeurs de `n_jobs` sur une machine coûte cher dès que chaque entraînement dure plusieurs dizaines de secondes. Le benchmark adaptatif réduit le nombre de configurations en exploitant la convexité habituelle de la courbe *temps vs n_jobs*. La phase 1 teste trois points : `n_jobs` minimal, médian et maximal. La phase 2 évalue les voisins immédiats du meilleur point. On s'arrête dès que l'amélioration relative passe sous un seuil $\varepsilon = 5\%$: $(T_{\text{best}} - T_{\text{nouveau}})/T_{\text{best}} < \varepsilon$.

2 Implémentation

L'implémentation s'appuie sur `RandomForestClassifier` de `scikit-learn`, qui prend un paramètre `n_jobs` contrôlant le nombre de workers. En interne, `scikit-learn` utilise `joblib` pour distribuer la construction des arbres de la forêt. Dans le cas des forêts aléatoires, cette parallélisation se fait typiquement par `threads`, car le cœur du calcul des arbres est écrit en code compilé `Cython` et libère le `GIL` (*Global Interpreter Lock*). Le parallélisme est donc réel malgré `CPython`.

Chaque worker construit une partie des B arbres de manière indépendante, puis le modèle final agrège les arbres obtenus. L'overhead vient donc de plusieurs sources : l'ordonnancement des tâches par `joblib`, les accès mémoire, les effets de cache, la synchronisation finale et l'agrégation des arbres. Cet overhead explique pourquoi augmenter `n_jobs` n'améliore pas toujours le temps d'entraînement.

2.1 Fonction de benchmark

L'extrait suivant mesure le temps d'entraînement pour une valeur donnée de `n_jobs`, répété pour réduire la variance.

```
def benchmark_random_forest(X_train, X_test, y_train, y_test,
                             n_jobs, n_estimators=200, n_repeats=3):
    times, accuracies = [], []
    for i in range(n_repeats):
        model = RandomForestClassifier(n_estimators=n_estimators,
                                       n_jobs=n_jobs, random_state=42 + i)
        start = time.perf_counter()
        model.fit(X_train, y_train)
        end = time.perf_counter()
        times.append(end - start)
        accuracies.append(accuracy_score(y_test, model.predict(X_test)))
    return {"n_jobs": n_jobs, "mean_time": np.mean(times),
            "std_time": np.std(times), "mean_accuracy": np.mean(accuracies)}
```

Listing 1 – Fonction de benchmark.

On utilise `time.perf_counter()` plutôt que `time.time()` pour sa meilleure résolution et son insensibilité aux ajustements d'horloge système. Faire varier la `random_state` entre les répétitions permet de moyenner sur plusieurs forêts bootstrapées et d'éviter de dépendre d'un seul tirage aléatoire. Les répétitions réduisent aussi la variance des mesures de temps.

2.2 Algorithme adaptatif

L'extrait ci-dessous correspond à la phase de raffinement : après les trois points initiaux, on évalue les voisins du meilleur `n_jobs` et on arrête dès que l'amélioration devient marginale.

```
def adaptive_search(benchmark_function, candidates,
                    initial_points=None, improvement_threshold=0.05):
    tested, results = set(), []
    for n_jobs in initial_points:
        result = benchmark_function(n_jobs)
        results.append(result); tested.add(n_jobs)
    best = min(results, key=lambda x: x["mean_time"])
    best_index = candidates.index(best["n_jobs"])
    for idx in [best_index-2, best_index-1, best_index+1, best_index+2]:
        if 0 <= idx < len(candidates) and candidates[idx] not in tested:
            result = benchmark_function(candidates[idx])
            results.append(result); tested.add(candidates[idx])
            current = min(results, key=lambda x: x["mean_time"])
            improvement = (best["mean_time"] - current["mean_time"]) / best["mean_time"]
            if improvement > improvement_threshold:
                best = current
    return results, min(results, key=lambda x: x["mean_time"])
```

Listing 2 – Phase de raffinement de la recherche adaptative.

3 Résultats expérimentaux

3.1 Protocole

Les expériences utilisent `make_classification` de scikit-learn pour générer 20 000 échantillons et 50 features (30 informatives, 10 redondantes), répartis 80/20 entre entraînement et test. Chaque forêt comporte 200 arbres, et chaque configuration est mesurée 3 fois.

3.2 Forêt vs arbre seul

Avant la parallélisation, on compare un arbre seul (profondeur 5) à une forêt de 200 arbres de même profondeur. L'arbre seul atteint 74,88% de précision, contre 85,85% pour la forêt, soit un gain de +10,97 points. Le bagging entraîne chaque arbre sur un bootstrap différent, leurs erreurs sont en partie décorréées, et le vote majoritaire lisse les sur-ajustements individuels. La matrice de confusion (annexe B) donne un taux d'erreur d'environ 14%, assez équilibré entre les deux classes, ce qui confirme que le jeu de test est symétrique. Cette comparaison utilise une profondeur limitée à 5 pour illustrer l'effet du bagging ; les benchmarks de temps ci-dessous utilisent une forêt non limitée en profondeur, ce qui explique l'accuracy plus élevée.

3.3 Mesures de parallélisation

Le tableau 1 rassemble les temps mesurés pour `n_jobs` de 1 à 8. Le temps tombe vite de 28,20s à 9,51s entre 1 et 4 workers, puis la décroissance ralentit. Au-delà de 4 workers, chaque processus supplémentaire apporte un gain marginal, et passer de 7 à 8 workers fait remonter le temps moyen : signe d'un overhead qui devient comparable au gain de parallélisme.

TABLE 1 – Temps, speedup et efficacité en fonction de `n_jobs` (200 arbres, 3 répétitions).

<code>n_jobs</code>	Temps (s)	Std (s)	Speedup	Eff.
1	28,20	0,39	1,00	1,000
2	15,54	0,25	1,81	0,907
3	10,80	0,09	2,61	0,870
4	9,51	0,21	2,96	0,741
5	8,60	0,30	3,28	0,656
6	8,59	1,01	3,28	0,547
7	6,70	0,02	4,21	0,601
8	6,95	0,82	4,06	0,507

La figure 1 compare les deux stratégies, et le tableau 2 synthétise leur coût d'exploration.

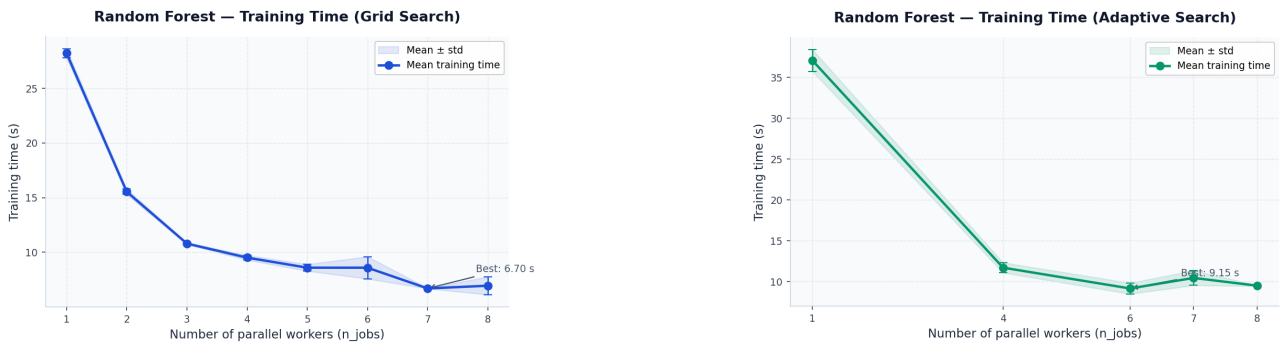


FIGURE 1 – Temps moyen ($\pm$ std) vs `n_jobs`. Gauche : grid search (8 configs). Droite : adaptive search (5 configs).

TABLE 2 – Grid search vs recherche adaptative.

Méthode	Configs	Best n	Temps (s)	Acc.
Grid search	8	7	6,70	94,8%
Adaptive search	5	6	9,15	94,8%

L'adaptive search retrouve un `n_jobs` proche de l'optimum (6 au lieu de 7) en testant 37,5% de configurations en moins. Le temps obtenu est plus élevé car, dans cette exécution séparée, la mesure de `n_jobs`=7 est plus bruitée et moins favorable que dans la grid search. La précision finale reste identique : `n_jobs` change l'exécution parallèle, pas l'algorithme statistique appris.

3.4 Speedup et loi d'Amdahl

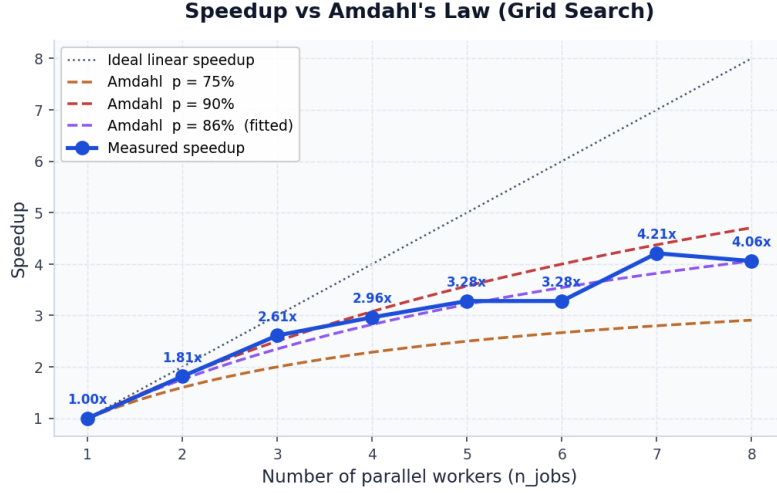


FIGURE 2 – Speedup mesuré contre courbes théoriques d'Amdahl pour différentes fractions parallèles p .

À partir du point $(n = 7, S = 4,21)$, on inverse la loi d'Amdahl : $p = n(S - 1)/(S(n - 1)) \approx 0,889$. En utilisant plutôt le point à 8 workers, on obtient $p \approx 0,861$. Les mesures suggèrent donc une fraction parallèle effective de l'ordre de 87–89%, et non un parallélisme parfait. Le plafond théorique associé est compris entre environ 7,2× et 9,1×, mais l'overhead pratique de `joblib`, les accès mémoire et l'agrégation finale limitent le gain observé bien avant.

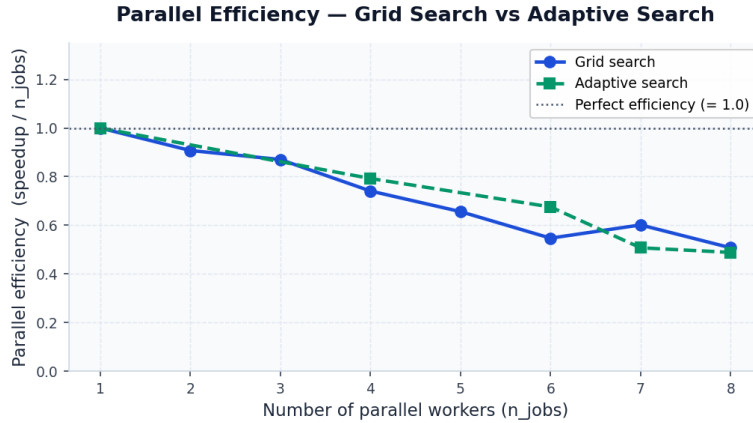


FIGURE 3 – Efficacité parallèle $E(n) = S(n)/n$ pour la grid search et la recherche adaptative.

La figure 3 confirme visuellement cette dégradation : l'efficacité passe de 1,0 avec 1 worker à environ 0,50 avec 8 workers, donc la moitié du temps CPU cumulé est perdue en overhead à 8 workers.

4 Conclusion

Trois points ressortent. D'abord, l'agrégation par bagging apporte un vrai gain de précision : +10,97 points entre l'arbre seul et la forêt à profondeur identique, ce qui justifie le coût de calcul plus élevé. Ensuite, la parallélisation est efficace mais bornée. Avec $p \approx 87\text{--}89\%$, la loi d'Amdahl prédit un speedup plafond d'environ 7 à 9×, mais en pratique on s'arrête vers 4,2× à 7 workers. La différence vient de l'overhead de `joblib` et du backend parallèle : ordonnancement des tâches, accès mémoire, effets de cache, synchronisation finale et agrégation des arbres.

L'optimum n'est donc pas `n_jobs` maximal : sur cette machine, le minimum est à 7 workers, et passer à 8 fait remonter le temps. La valeur optimale dépend de la machine et du dataset, elle ne peut être trouvée que par mesure. Enfin, le benchmark adaptatif retrouve un optimum proche en testant 37,5% de configurations en moins. C'est utile quand on calibre un pipeline sur plusieurs machines ou plusieurs datasets, où chaque mesure coûte cher.

Annexe A – Illustration de l’algorithme CART

La figure 4 montre un arbre extrait de notre forêt avec une profondeur limitée à 3 pour la lisibilité. Chaque nœud interne affiche la condition $x_j \leq t^*$ choisie par CART, l’impureté de Gini calculée selon l’équation (1), le nombre d’échantillons et la distribution des classes. Les classes se séparent au fur et à mesure de la descente, et la profondeur limite la complexité du modèle.

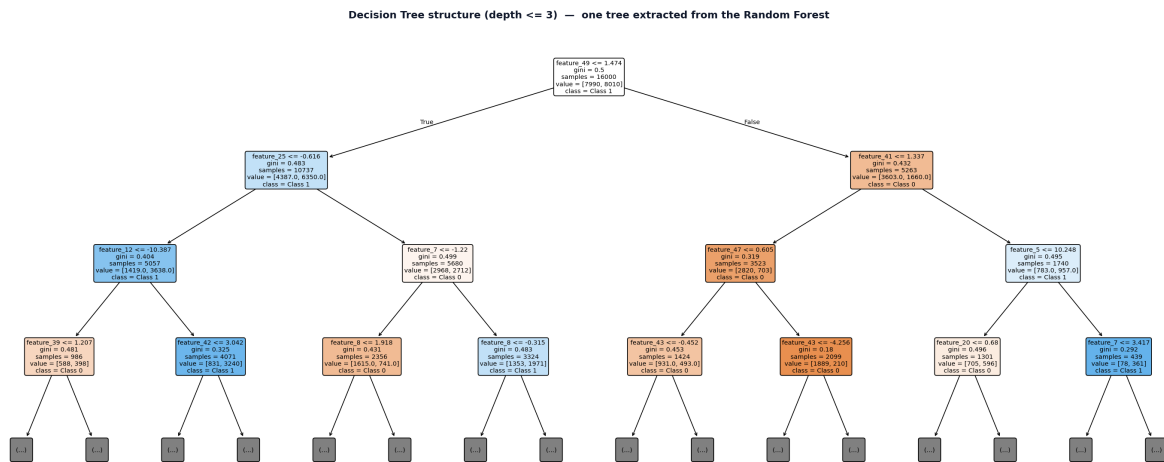


FIGURE 4 – Arbre de décision extrait de la forêt (profondeur 3). Chaque nœud affiche la condition de split, l’impureté de Gini, le nombre d’échantillons et la distribution des classes. Les feuilles colorées correspondent à la classe prédite.

Annexe B – Interprétabilité et matrice de confusion

La figure 5 donne les 15 features les plus discriminantes selon notre forêt. L’importance d’une feature est la diminution moyenne d’impureté de Gini qu’elle apporte, agrégée sur les 200 arbres et pondérée par le nombre d’échantillons concernés. Une feature très utilisée près de la racine accumule donc une forte importance. Les barres d’erreur représentent l’écart-type inter-arbres, qui mesure la stabilité de l’estimation. Comme les features sont synthétiques, elles n’ont pas de sens métier ici, mais peu d’entre elles concentrent l’essentiel du pouvoir prédictif, ce qui est typique des problèmes réels.

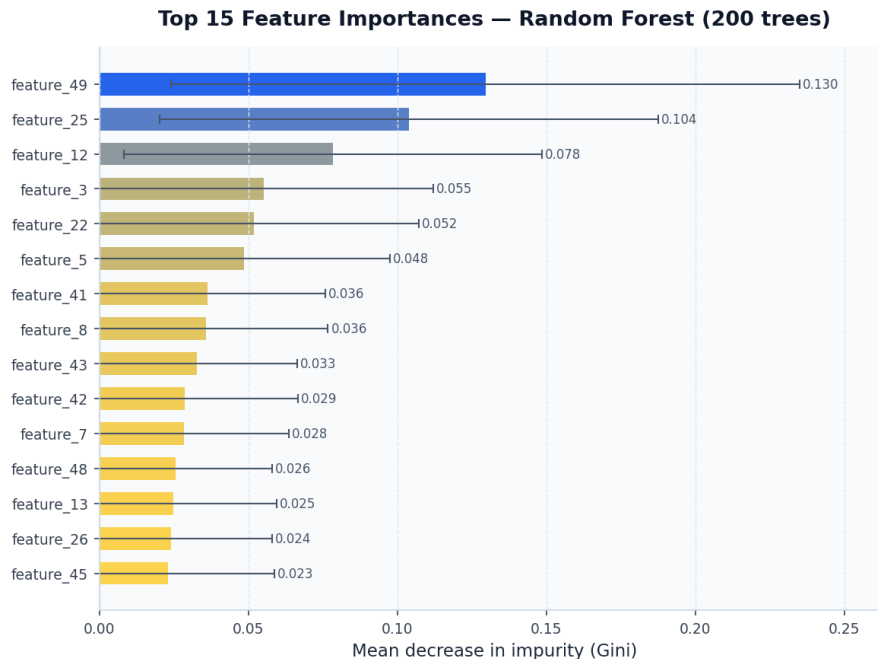


FIGURE 5 – Importance des 15 features les plus discriminantes (diminution moyenne d’impureté de Gini sur 200 arbres). Barres d’erreur : écart-type inter-arbres.

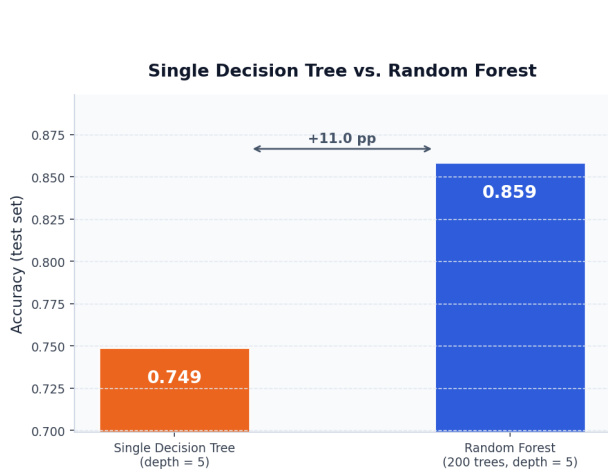


FIGURE 6 – Arbre seul vs forêt de 200 arbres (profondeur 5). Gain de +10,97 pp dû à la réduction de variance par bagging.

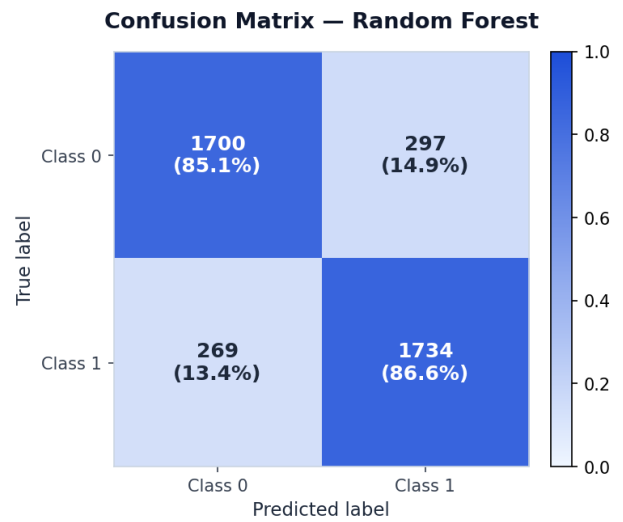


FIGURE 7 – Matrice de confusion de la forêt sur 4 000 échantillons de test. Le taux d’erreur d’environ 14% est assez équilibré entre les classes.

Annexe C – Discussion approfondie

Limites de la loi d’Amdahl. La fraction séquentielle effective augmente en réalité avec n , parce que l’ordonnancement des tâches, la pression mémoire et les effets de cache deviennent plus coûteux quand le nombre de workers augmente. La loi d’Amdahl reste un bon premier ordre, mais une modélisation plus fine (loi de Gustafson, modèles BSP) serait nécessaire pour prédire précisément le speedup au-delà de 4-5 workers. La loi de Gustafson, en particulier, suppose que la taille du problème croît avec n , ce qui décrit mieux les usages réels en machine learning, où des datasets plus gros tournent sur plus de cœurs.

$n_jobs=-1$ en pratique. Utiliser $n_jobs=-1$ (tous les cœurs) par défaut, comme on le voit souvent en pratique, n’est généralement pas le meilleur choix : sur nos mesures, 7 workers battent 8 alors que la machine a 8 cœurs logiques. Cela montre l’importance d’un benchmark systématique avant tout déploiement, surtout sur des machines avec hyperthreading où les cœurs logiques ne valent pas les cœurs physiques.

Extension du benchmark adaptatif. La stratégie de raffinement local risquerait de manquer l’optimum global face à une courbe à plusieurs minima. Une extension naturelle serait de combiner cette recherche avec une exploration aléatoire des candidats restants après convergence locale, par exemple une stratégie hybride inspirée du *simulated annealing*.